Sesión 14

Despliegue de Modelos (MLOps): De Jupyter Notebook a producción

Carlos César Sánchez Coronel

MACHINE LEARNING

Contenido

1. Introducción a MLOps
2. Serialización de Modelos
3. Creación de APIs con Flask y FastAPI
4. Contenerización con Docker
5. Versionamiento de Modelos y Datos
6. Monitoreo de Modelos en Producción
7. Estrategias de Despliegue y A/B Testing
8. Caso Integrado: Modelo de Riesgo Crediticio
9. Revisión General del Curso
10. Conexión con el Futuro Profesional
11. Resumen y Conclusión

Contenido

1. Introducción a MLOps
2. Serialización de Modelos
3. Creación de APIs con Flask y FastAPI
4. Contenerización con Docker
5. Versionamiento de Modelos y Datos
6. Monitoreo de Modelos en Producción
7. Estrategias de Despliegue y A/B Testing
8. Caso Integrado: Modelo de Riesgo Crediticio
9. Revisión General del Curso
10. Conexión con el Futuro Profesional
11. Resumen y Conclusión

El último kilómetro del machine learning

- Construir un modelo en Jupyter es solo el 10 % del trabajo.
- El 90 % restante es llevarlo a producción y mantenerlo.
- **MLOps**: disciplina que combina ML, ingeniería de software y DevOps.
- Objetivos: disponibilidad, escalabilidad, reproducibilidad, monitoreo continuo.

Entrenamiento vs Inferencia

Entrenamiento

- Offline, por lotes.
- Puede ser lento (horas/días).
- Grandes volúmenes de datos.
- Poco frecuente.

Inferencia

- Online (tiempo real) o batch.
- Baja latencia (online).
- Alta concurrencia.
- Continua.

La arquitectura debe separar ambos pipelines.

Contenido

1. Introducción a MLOps
2. Serialización de Modelos
3. Creación de APIs con Flask y FastAPI
4. Contenerización con Docker
5. Versionamiento de Modelos y Datos
6. Monitoreo de Modelos en Producción
7. Estrategias de Despliegue y A/B Testing
8. Caso Integrado: Modelo de Riesgo Crediticio
9. Revisión General del Curso
10. Conexión con el Futuro Profesional
11. Resumen y Conclusión

Formatos de serialización

Pickle

- Estándar de Python.
- Inseguro, problemas de compatibilidad.

Joblib

- Optimizado para arrays NumPy.
- Recomendado para scikit-learn.

ONNX

- Formato abierto e interoperable.
- Permite optimizaciones de inferencia.

MLflow: plataforma para gestionar modelos, con registro y versionado.

Ejemplos de serialización

```
import pickle
with open('modelo.pkl', 'wb') as f:
    pickle.dump(model, f)

import joblib
joblib.dump(model, 'modelo.joblib')
model = joblib.load('modelo.joblib')

# ONNX (skl2onnx)
from skl2onnx import convert_sklearn
from skl2onnx.common.data_types import FloatTensorType
initial_type = [('float_input', FloatTensorType([None, X.shape[1]]))]
onx = convert_sklearn(model, initial_types=initial_type)
with open("model.onnx", "wb") as f:
    f.write(onx.SerializeToString())
```

Contenido

1. Introducción a MLOps
2. Serialización de Modelos
3. Creación de APIs con Flask y FastAPI
4. Contenerización con Docker
5. Versionamiento de Modelos y Datos
6. Monitoreo de Modelos en Producción
7. Estrategias de Despliegue y A/B Testing
8. Caso Integrado: Modelo de Riesgo Crediticio
9. Revisión General del Curso
10. Conexión con el Futuro Profesional
11. Resumen y Conclusión

Flask vs FastAPI

Característica	**Flask**	**FastAPI**
Rendimiento	Bajo	Alto (asíncrono)
Documentación automática	No	Sí (Swagger, ReDoc)
Validación de datos	Con extensiones	Integrada (Pydantic)
Soporte asíncrono	Limitado	Nativo

Ejemplo con Flask

```python
from flask import Flask, request, jsonify
import joblib

app = Flask(__name__)
model = joblib.load('modelo.joblib')

@app.route('/predict', methods=['POST'])
def predict():
    data = request.get_json()
    features = data['features']
    pred = model.predict([features])
    return jsonify({'prediction': pred.tolist()})

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

Ejemplo con FastAPI

```python
from fastapi import FastAPI
from pydantic import BaseModel
import joblib

app = FastAPI()
model = joblib.load('modelo.joblib')

class Item(BaseModel):
    features: list

@app.post('/predict')
def predict(item: Item):
    pred = model.predict([item.features])
    return {'prediction': pred.tolist()}
```

- Documentación interactiva en /docs.
- Validación automática con Pydantic.

Contenido

1. Introducción a MLOps
2. Serialización de Modelos
3. Creación de APIs con Flask y FastAPI
4. Contenerización con Docker
5. Versionamiento de Modelos y Datos
6. Monitoreo de Modelos en Producción
7. Estrategias de Despliegue y A/B Testing
8. Caso Integrado: Modelo de Riesgo Crediticio
9. Revisión General del Curso
10. Conexión con el Futuro Profesional
11. Resumen y Conclusión

¿Por qué Docker?

- Elimina el problema de “funciona en mi máquina”.
- Empaqueta la aplicación con todas sus dependencias.
- Misma ejecución en cualquier entorno.

Dockerfile para una API de modelo

```
FROM python:3.9-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install -r requirements.txt

COPY model.joblib .
COPY app.py .

CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "80"]
```

- Construir: `docker build -t modelo-api .`
- Ejecutar: `docker run -p 80:80 modelo-api`

Contenido

1. Introducción a MLOps
2. Serialización de Modelos
3. Creación de APIs con Flask y FastAPI
4. Contenerización con Docker
5. Versionamiento de Modelos y Datos
6. Monitoreo de Modelos en Producción
7. Estrategias de Despliegue y A/B Testing
8. Caso Integrado: Modelo de Riesgo Crediticio
9. Revisión General del Curso
10. Conexión con el Futuro Profesional
11. Resumen y Conclusión

MLflow para seguimiento de experimentos

Componentes principales

- **Tracking**: registro de parámetros, métricas, artefactos.
- **Projects**: empaquetado de código.
- **Models**: formato estándar para modelos.
- **Model Registry**: almacén centralizado con versionado y transiciones (staging, production).

Ejemplo con MLflow

```
import mlflow

with mlflow.start_run():
    mlflow.log_param("n_estimators", 100)
    mlflow.log_metric("auc", 0.85)
    mlflow.sklearn.log_model(model, "model")

# Cargar una versi n espec fica
model = mlflow.pyfunc.load_model("models:/credit_model/1")
```

DVC (Data Version Control)

- Versiona datasets y modelos (archivos grandes).
- Integrado con Git: Git guarda metadatos, DVC maneja el almacenamiento remoto.
- Flujo: `dvc add data.csv` → `git add data.csv.dvc` → `git commit` → `dvc push`

Contenido

1. Introducción a MLOps
2. Serialización de Modelos
3. Creación de APIs con Flask y FastAPI
4. Contenerización con Docker
5. Versionamiento de Modelos y Datos
6. Monitoreo de Modelos en Producción
7. Estrategias de Despliegue y A/B Testing
8. Caso Integrado: Modelo de Riesgo Crediticio
9. Revisión General del Curso
10. Conexión con el Futuro Profesional
11. Resumen y Conclusión

Data drift y concept drift

- **Data drift**: cambio en la distribución de las variables de entrada.
- **Concept drift**: cambio en la relación entrada-salida.
- Causas: cambios en el mercado, nuevos hábitos, crisis económicas.

Population Stability Index (PSI)

$$PSI = \sum_{i=1}^{B} (p_i - q_i) \cdot \ln\left(\frac{p_i}{q_i}\right)$$

donde p_i = proporción en producción, q_i = proporción en entrenamiento.

- $PSI < 0{,}1$: cambio pequeño.
- $0{,}1 \leq PSI < 0{,}25$: cambio moderado.
- $PSI \geq 0{,}25$: cambio significativo (reentrenar).

Cálculo de PSI en Python

```
import numpy as np

def psi(expected, actual, bins=10):
    expected = np.clip(expected, 0.001, 0.999)  # evitar log(0)
    actual = np.clip(actual, 0.001, 0.999)
    return np.sum((actual - expected) * np.log(actual / expected))
```

Test de Kolmogorov-Smirnov (KS)

- Compara dos distribuciones continuas sin discretizar.
- Estadístico: máxima diferencia entre las funciones de distribución acumulada.
- p-valor bajo $\Rightarrow$ distribuciones diferentes.
- Útil para variables continuas.

Herramientas de monitoreo

- **Evidently AI**: reportes de drift, calidad de datos, rendimiento.
- **WhyLabs / Whylogs**: perfilado de datos y detección de drift.
- **Great Expectations**: validación de datos (expectativas).
- **Prometheus + Grafana**: métricas del sistema.

Contenido

1. Introducción a MLOps
2. Serialización de Modelos
3. Creación de APIs con Flask y FastAPI
4. Contenerización con Docker
5. Versionamiento de Modelos y Datos
6. Monitoreo de Modelos en Producción
7. Estrategias de Despliegue y A/B Testing
8. Caso Integrado: Modelo de Riesgo Crediticio
9. Revisión General del Curso
10. Conexión con el Futuro Profesional
11. Resumen y Conclusión

A/B Testing para modelos en producción

¿Qué es un A/B test?

Experimento controlado donde se comparan dos versiones de un modelo (A = control, B = tratamiento) asignando usuarios aleatoriamente.

- **Objetivo**: determinar si el nuevo modelo (B) mejora métricas de negocio respecto al actual (A).
- **Asignación aleatoria**: asegura que los grupos sean comparables.
- **Métrica primaria**: una única métrica clave (ej. tasa de conversión, ingresos por usuario).

Diseño de un A/B test para modelos

1. **Definir la hipótesis**: ej. ."el nuevo modelo de recomendación aumenta el tiempo de visualización en un 5 %".
2. **Elegir métricas**: primaria (tiempo de visualización) y secundarias (clic, retención).
3. **Calcular tamaño de muestra** necesario para significancia estadística (poder, nivel de confianza).
4. **Asignar usuarios aleatoriamente** (por ejemplo, mediante hash de usuario).
5. **Ejecutar el experimento** durante un período representativo (evitar estacionalidades).
6. **Analizar resultados**: test estadístico (t-test, bootstrap) y significancia práctica (no solo p-valor).

Métricas de negocio vs técnicas

Métricas técnicas

- Precisión, recall, AUC
- RMSE, MAE
- Latencia, throughput

Métricas de negocio

- Tasa de conversión
- Ingresos por usuario
- Retención / churn
- Tiempo de sesión

Importante

Una mejora técnica no siempre se traduce en mejora de negocio. El A/B test valida el impacto real.

Ejemplo: comparar dos modelos de recomendación

Contexto

Plataforma de streaming quiere probar un nuevo algoritmo de recomendación.

- Grupo A (control): modelo actual (basado en popularidad).
- Grupo B (tratamiento): nuevo modelo (filtrado colaborativo).
- Duración: 2 semanas, 100,000 usuarios por grupo.
- Métrica primaria: tiempo medio de visualización por usuario.
- Resultado: B muestra un aumento del 3 % con p-valor ¡ 0.01.
- Decisión: desplegar B gradualmente (canary).

Precauciones estadísticas

- **Significancia estadística**: usar p-valor (ej. ¡ 0.05) y considerar corrección por múltiples comparaciones si hay varias métricas.
- **Tamaño de muestra**: calcular previamente para evitar falsos negativos (poder estadístico).
- **Efecto mínimo detectable**: definir qué mejora es relevante para el negocio.
- **Duración**: suficiente para capturar variabilidad (evitar sesgos por día de la semana).
- **Interferencia entre grupos**: asegurar que los grupos no interactúen (ej. red social).

Integración con despliegue canario

- El A/B test puede ser la primera fase de un despliegue canario.
- **Canario**: enrutar un pequeño
- **Medir** las mismas métricas de negocio durante el canario.
- Si los resultados son positivos, aumentar gradualmente el porcentaje.
- Si no, revertir automáticamente (rollback).

Contenido

1. Introducción a MLOps
2. Serialización de Modelos
3. Creación de APIs con Flask y FastAPI
4. Contenerización con Docker
5. Versionamiento de Modelos y Datos
6. Monitoreo de Modelos en Producción
7. Estrategias de Despliegue y A/B Testing
8. Caso Integrado: Modelo de Riesgo Crediticio
9. Revisión General del Curso
10. Conexión con el Futuro Profesional
11. Resumen y Conclusión

Paso 1: Entrenar y guardar el modelo

```
df = pd.read_csv(
'credito.csv') X = df.drop('incumple', axis=1) y = df['incumple'] model =
    RandomForestClassifier(n_estimators = 100)model.fit(X, y)joblib.dump(model, '../credit_model.joblib')
```

Paso 2: Crear API con FastAPI

```python
from fastapi import FastAPI
from pydantic import BaseModel
import joblib
import numpy as np

app = FastAPI()
model = joblib.load('credit_model.joblib')

class Applicant(BaseModel):
    edad: int
    ingresos: float
    deuda: float
    antiguedad_laboral: int

@app.post('/predict')
def predict(applicant: Applicant):
    features = np.array([[applicant.edad, applicant.ingresos,
                          applicant.deuda, applicant.antiguedad_laboral]])
    proba = model.predict_proba(features)[0, 1]
    return {'probabilidad_incumplimiento': proba}
```

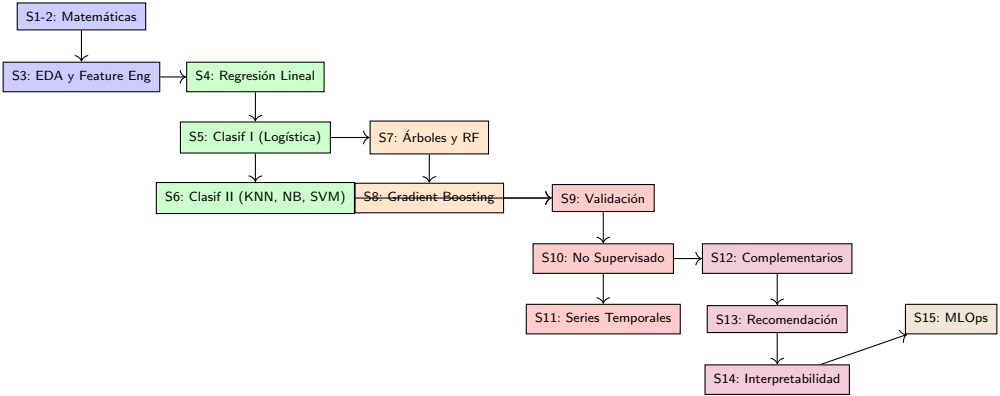
Pasos 3-5: Docker, MLflow y monitoreo

- **Docker**: contenerizar la API con el Dockerfile visto.
- **MLflow**: registrar el modelo y versionarlo.
- **Monitoreo**: calcular PSI semanal de las características y predicciones; alertar si PSI ¿ 0.25.

Contenido

1. Introducción a MLOps
2. Serialización de Modelos
3. Creación de APIs con Flask y FastAPI
4. Contenerización con Docker
5. Versionamiento de Modelos y Datos
6. Monitoreo de Modelos en Producción
7. Estrategias de Despliegue y A/B Testing
8. Caso Integrado: Modelo de Riesgo Crediticio
9. Revisión General del Curso
10. Conexión con el Futuro Profesional
11. Resumen y Conclusión

Mapa conceptual del curso



Contenido

1. Introducción a MLOps
2. Serialización de Modelos
3. Creación de APIs con Flask y FastAPI
4. Contenerización con Docker
5. Versionamiento de Modelos y Datos
6. Monitoreo de Modelos en Producción
7. Estrategias de Despliegue y A/B Testing
8. Caso Integrado: Modelo de Riesgo Crediticio
9. Revisión General del Curso
10. Conexión con el Futuro Profesional
11. Resumen y Conclusión

Más allá del curso

- Dominar MLOps diferencia a un junior de un senior.
- Habilidades valoradas: despliegue, monitoreo, CI/CD, escalabilidad.
- Herramientas a explorar: Kubeflow, Airflow, MLflow, DVC, Kubernetes.
- Práctica continua, lectura de papers, competencias (Kaggle).
- Mantenerse actualizado con nuevas técnicas.

Contenido

1. Introducción a MLOps
2. Serialización de Modelos
3. Creación de APIs con Flask y FastAPI
4. Contenerización con Docker
5. Versionamiento de Modelos y Datos
6. Monitoreo de Modelos en Producción
7. Estrategias de Despliegue y A/B Testing
8. Caso Integrado: Modelo de Riesgo Crediticio
9. Revisión General del Curso
10. Conexión con el Futuro Profesional
11. Resumen y Conclusión

Lo que hemos aprendido

Conceptos clave

- MLOps: integración de ML con operaciones.
- Serialización: pickle, joblib, ONNX, MLflow.
- APIs: Flask, FastAPI (recomendado).
- Contenerización: Docker.
- Versionado: MLflow (modelos), DVC (datos).
- Monitoreo: PSI, KS, herramientas (Evidently, Prometheus).
- Estrategias de despliegue: canary, blue-green.
- A/B testing y seguridad.

Conclusión del curso

- Recorrido completo: de matemáticas a producción.
- Modelos supervisados, no supervisados, series temporales, recomendación.
- Interpretabilidad y MLOps como cierre.
- El aprendizaje continúa: práctica, actualización, comunidad.

¡Enhorabuena por llegar hasta el final!

¡Gracias!